

Rapport SAE 2-01

1. Introduction

Dans une démarche de transformation numérique, une banque de Clermont a engagé un chantier de refonte de son système de gestion des données. Jusqu'à présent, les informations relatives à ses clients, agents, comptes et opérations étaient stockées dans un fichier CSV simple, peu adapté à une montée en charge ou à une gestion fiable sur le long terme.

Pour répondre aux enjeux de sécurité, traçabilité et évolutivité, la banque a exprimé le besoin de concevoir une base de données relationnelle robuste, structurée autour du SGBD PostgreSQL. Le système cible doit permettre :

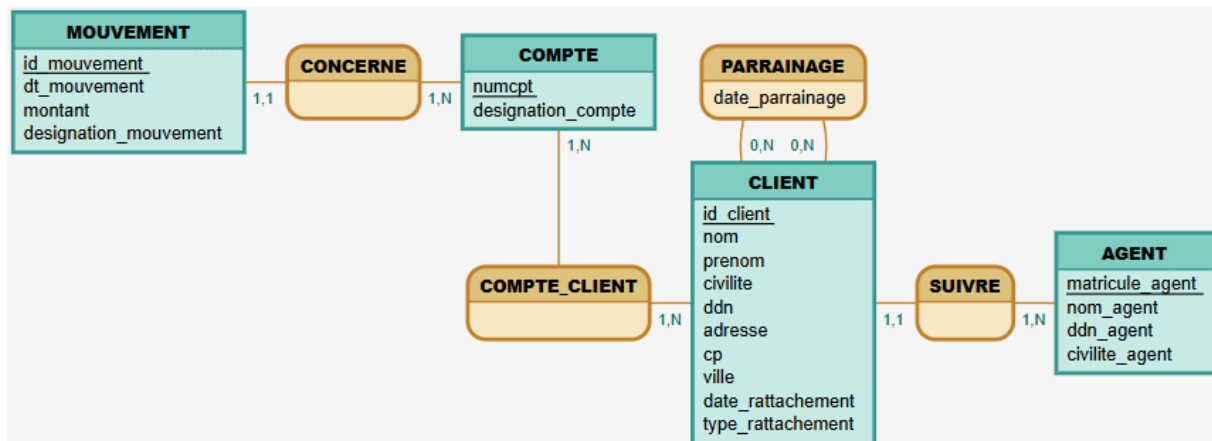
- Une gestion fine des clients et de leur rattachement à un agent référent,
- La modélisation des comptes bancaires et de leur multi-détention éventuelle,
- L'historisation des mouvements financiers (crédits/débits),
- Le suivi des relations de parrainage entre clients.

2. MCD

Le Modèle conceptuel de données (MCD) sert de plan directeur : il décrit de façon abstraite les objets manipulés par l'application et la manière dont ils interagissent. Pour le bâtir, j'ai choisi Mocodo Online qui est un outil que nous avons utilisé en classe et qui permet de facilement visualiser ce MCD.

Voici les entités et associations que j'ai retenues pour faire mon MCD :

CLIENT	Personne physique ou morale titulaire d'au moins un compte
AGENT	Conseiller référent chargé du suivi des clients
COMPTE	Support sur lequel sont enregistrés les opérations financières
MOUVEMENT	Trace d'un crédit ou d'un débit lié à un compte
(associations)	POSSEDER (relation n : n CLIENT ↔ COMPTE) et PARRAINER (auto-relation CLIENT ↔ CLIENT)



3. MLD

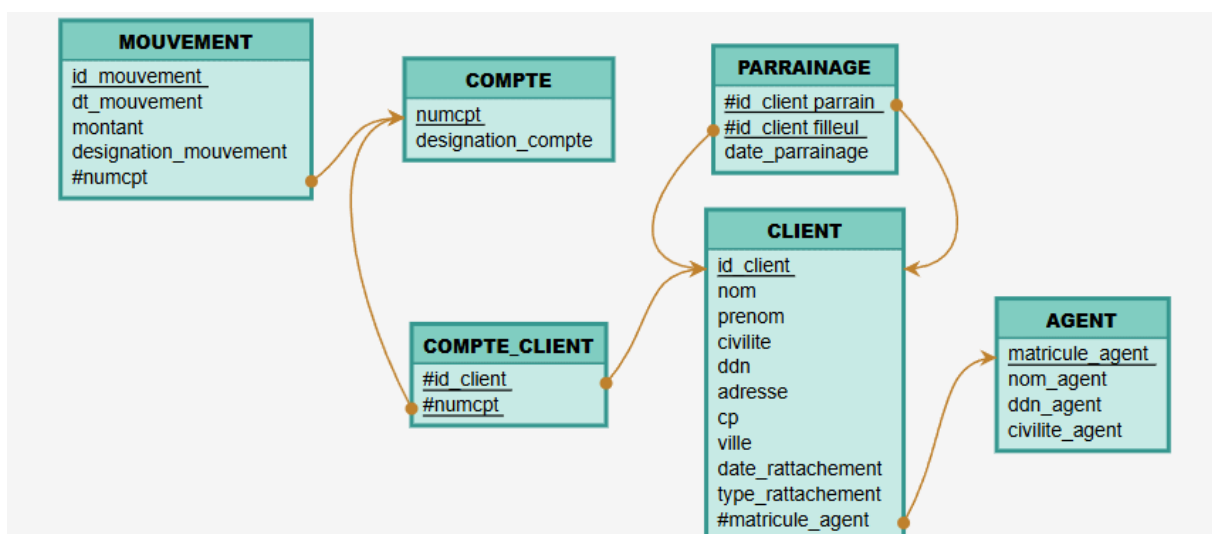
Une fois le MCD établi, le passage au Modèle Logique de Données (MLD) s'effectue naturellement. Grâce à Mocodo, cette étape a été en grande partie automatisée : chaque entité du MCD est transformée en une table relationnelle, avec ses attributs, sa clé primaire et les éventuelles clés étrangères reliant les entités entre elles.

Les entités comme CLIENT, AGENT, COMPTE ou MOUVEMENT deviennent directement des tables : les attributs sont repris sans altération, en cohérence avec les noms issus du CSV (numcpt, dt_mouvement, ddn, etc.).

Les **associations** telles que :

- COMPTE_CLIENT (relation plusieurs-à-plusieurs entre CLIENT et COMPTE)
- PARRAINAGE (auto-relation de CLIENT à CLIENT)

sont transformées en **tables de jointure** dotées de leurs propres clés primaires composites et éventuellement d'attributs spécifiques (comme date_parrainage).



4. Nettoyage des données

Avant toute insertion en base, un nettoyage préalable du fichier CSV a été nécessaire. Le jeu de données d'origine contenait en effet plusieurs incohérences sur les colonnes de type date, notamment des valeurs invalides ou non interprétables par les moteurs de base de données.

Parmi les cas problématiques identifiés :

- Des dates impossibles, comme le 29 février 2017 (année non bissextile),
- Des champs partiellement remplis (31/04, 00/00/0000, etc.),
- Des mélanges de formats (AAAA-MM-JJ et JJ/MM/AAAA coexistaient).

Pour garantir la qualité des données, un script Python a été conçu. Il s'appuie sur la bibliothèque pandas, combinée à une gestion fine des erreurs et que toutes les dates soient désormais sous le même format JJ/MM/AAAA.

5. Script SQL, Insertion des données et Vues

A) Insertion du squelette de la base

L'ossature de la base a été bâtie à l'aide de **scripts SQL PostgreSQL** en tenant compte des dépendances hiérarchiques entre entités :

```
CREATE TABLE AGENT (  
    matricule_agent INT PRIMARY KEY,  
    nom_agent      TEXT,  
    ddn_agent      DATE,  
    civilite_agent CHAR(1) CHECK (civilite_agent IN ('M','F'))  
);
```

```
CREATE TABLE CLIENT (  
    id_client      INT PRIMARY KEY,  
    nom            TEXT,  
    prenom         TEXT,  
    civilite       CHAR(1) CHECK (civilite IN ('M','F')),  
    ddn            DATE,  
    adresse        TEXT,  
    cp             INT,  
    ville          TEXT,  
    date_rattachement DATE,
```

```
type_rattachement TEXT,  
matricule_agent INT REFERENCES AGENT(matricule_agent)  
);
```

```
CREATE TABLE COMPTE (  
    numcpt BIGINT PRIMARY KEY, -- BIGINT : certains numéros dépassent INT  
    designation_compte TEXT  
);
```

```
CREATE TABLE COMPTE_CLIENT (  
    id_client INT REFERENCES CLIENT(id_client),  
    numcpt BIGINT REFERENCES COMPTE(numcpt),  
    PRIMARY KEY (id_client, numcpt)  
);
```

```
CREATE TABLE MOUVEMENT (  
    id_mouvement INT PRIMARY KEY,  
    dt_mouvement DATE,  
    montant FLOAT,  
    designation_mouvement TEXT,  
    numcpt BIGINT REFERENCES COMPTE(numcpt)  
);
```

```
CREATE TABLE PARRAINAGE (  
    id_client_parrain INT REFERENCES CLIENT(id_client),  
    id_client_filleul INT REFERENCES CLIENT(id_client),  
    date_parrainage DATE,  
    PRIMARY KEY (id_client_parrain, id_client_filleul)  
);
```

Pourquoi BIGINT ?

Les numéros de compte provenant du CSV dépassaient ponctuellement la limite d'un INTEGER : l'usage de BIGINT supprime tout risque de dépassement.

B) Chargement des données brutes

Une table temporaire, TABLE_TEMPORAIRE reprend toutes les colonnes du fichier nettoyé :

```
\copy TABLE_TEMPORAIRE FROM 'sae_donnees_bancaires_nettoyee.csv' DELIMITER ','  
CSV HEADER;
```

Cette étape introduit un **sas de sécurité** : aucune contrainte n'est encore appliquée, ce qui évite de bloquer le chargement initial.

C) Dispatch des données vers les tables cibles

Les informations utiles sont ensuite redistribuées **table par table**, en appliquant DISTINCT pour éliminer les doublons :

```
INSERT INTO AGENT (matricule_agent, nom_agent, ddn_agent, civilite_agent)
SELECT DISTINCT matricule_agent, nom_agent, ddn_agent, civilite_agent
FROM TABLE_TEMPORAIRE;
```

```
INSERT INTO CLIENT (id_client, nom, prenom, civilite, ddn, adresse, cp, ville,
date_rattachement, type_rattachement, matricule_agent)
SELECT DISTINCT ON (id_client) id_client, nom, prenom, civilite, ddn, adresse, cp, ville,
date_rattachement, type_rattachement, matricule_agent
FROM TABLE_TEMPORAIRE;
```

```
INSERT INTO COMPTE (numcpt, designation_compte)
SELECT DISTINCT numcpt, designation_compte
FROM TABLE_TEMPORAIRE;
```

```
INSERT INTO COMPTE_CLIENT (id_client, numcpt)
SELECT DISTINCT id_client, numcpt
FROM TABLE_TEMPORAIRE;
```

```
INSERT INTO MOUVEMENT (id_mouvement, dt_mouvement, montant,
designation_mouvement, numcpt)
SELECT DISTINCT ON (id_mouvement) id_mouvement, dt_mouvement, montant,
designation_mouvement, numcpt
FROM TABLE_TEMPORAIRE
WHERE id_mouvement IS NOT NULL;
```

```
INSERT INTO PARRAINAGE (id_client_parrain, id_client_filleul, date_parrainage)
SELECT
    c_parrain.id_client AS id_client_parrain,
    tmp.id_client AS id_client_filleul,
    tmp.date_parrainage
FROM TABLE_TEMPORAIRE tmp
JOIN CLIENT c_parrain ON LOWER(c_parrain.nom) = LOWER(tmp.parrain)
```

WHERE tmp.parrain IS NOT NULL;

Chaque insertion déclenche **les contraintes d'intégrité** (PRIMARY KEY, FOREIGN KEY, CHECK) définies lors de la création du schéma, garantissant ainsi la cohérence des données importées.

D) Vues pour le dirigeant

Deux vues synthétiques répondent aux besoins de pilotage & leurs commandes d'export en csv associées :

1. **comptes_dormants** – listes des comptes sans activité depuis le début de l'année :
`\copy (SELECT * FROM comptes_dormants) TO 'comptes_dormants.csv' WITH CSV HEADER ;`
2. **etat_parrainages** – suivi des parrainages enregistrés depuis le 1^{er} janvier :
`\copy (SELECT * FROM etat_parrainages) TO 'etat_parrainages.csv' WITH CSV HEADER;`

6. Conclusion

Grâce à cette démarche :

- Modélisation, normalisation (3FN) et scripting SQL ont été menés de bout en bout.
- Les vues dynamiques fournissent aux décideurs une vision en temps réel, tout en restant simples à extraire en CSV.
- Les problèmes concrets rencontrés (types numériques, relations $n \leftrightarrow n$, qualité des dates) ont été résolus par une approche structurée combinant PostgreSQL et Python.

Le livrable final constitue ainsi une base bancaire pérenne, évolutive et conforme aux exigences fixées dans le cahier des charges de la SAÉ.